

Model-Driven Business Performance Management

Liangzhao Zeng, Hui Lei, Michael Dikun, Henry Chang, Kumar Bhaskaran

IBM T.J. Watson Research Center

lzeng,hlei,mdikun,hychang,bha@us.ibm.com

Joachim Frank

IBM Software Group

jhfrank@us.ibm.com

Abstract

In this paper, we present a model-driven approach to Business Performance Management (BPM). BPM is a new frontier in IT-enabled enterprise that supports the monitoring and control of business operations. BPM solutions must be able to efficiently process business events, compute business metrics, detect business situations, and provide the real-time visibility of key performance indicators. In addition, system support is required for the rapid development of BPM solutions and the adaptation of the solutions to the dynamic business environment. We have adopted a meta-model, dubbed the observation meta-model, for capturing the business requirements for BPM, which frees solution developers from low-level programming concerns. We have also used a hybrid compilation-interpretation approach to map an observation model to the runtime executable. First, we extract and refactor the data aspect of the observation model to facilitate runtime access. Second, we compile the operational aspect of the model, such as logic for metric computation and situation detection, into Java code. Third, we develop a runtime engine that interprets the refactored model and dynamically loads the generated code, according to the meta-model. Our framework further enables the evolution and hot deployment of the observation model and provides the platform for several on-going customer engagement efforts.

1 Introduction

In order to function effectively in today's business environment, organizations must have visibility in their business activities and operation performance at all times. This allows them to stay competitive and profitable. BPM is a new generation of enterprise data management system that focuses on monitoring business operations. It provides a

comprehensive view of business operation in organizations. There are some desired benefits when adopting the BPM solution, including:- (1) Increasing revenue by speeding response time, actions and regulatory changes; (2) Effectively managing risk by providing information in the right context to facilitate decision making; (3) Improving customer satisfaction by allowing continuous improvement of business processes.

In this project, we advocate model-driven BPM solutions. In general, a model provides an abstraction of a system that allow users to reason about that system without needing to understand extraneous details. In this paper, we define an *observation model* to be an abstraction of an observation system, which captures the requirements of BPM solutions. The observation model includes a collection of metrics that are used to measure the performance of business operations, which is declaratively defined by solution providers. In buildtime, after performing a series of transformations and processing with an observation model, the executable code is generated and then deployed into the runtime platform. In runtime, once the generated code is deployed, the runtime engine computes metric values in real-time by processing data from both live events and persistent datastores, and generates alerts once situations occur.

When a model-driven approach is adopted, building a BPM solution becomes centered around observations models. This approach differs from existing BPM solutions. In existing approaches, developers focus on time-consuming platform dependent implementation. While in our model-driven approach, solution providers can concentrate on business processes and observation models, without worrying about platform specific implementation details. Further, BPM solutions can be continuously improved by modifying the observation model, as the transformation and deployment performs systematically through the model-driven architecture.

Implementing a model-driven BPM is by no means a

trivial task. There are two major challenges of implementing model-driven BPM:-

- There are large numbers of the business entities that need to be monitored, and these business entities may be associated with an even larger number of metrics. Further, the data structure for describing metric may be complex. The challenge is to design a BPM runtime engine that is able to compute and maintain the large amount of metric data in a timely manner, wherein data source processed by the runtime engine can come from live events in addition to persistent datastores. The runtime engine is also required to efficiently track the state of metrics in order to detect business situations.
- BPM solutions are often associated with business processes. They need to adapt to changes in the processes. The continuous nature of business process instances requires the BPM runtime engine to operate continuously. This means that new models can be deployed without shutting down the runtime engine (i.e., hot deployment). Further, the migration of the observation model should be performed in synchronization with the progress of business process migration. In some case, live business process instances may belong to different versions of the process schemas. This means that multiple versions of the observation models may coexist in the runtime engine.

We proposed a novel hybrid compilation-interpretation to implement the model-driven architecture, which aims for efficient and timely management of business performance. Further, we design a mechanism for managing dynamic evolution of observation models. The major salient features and contributions of our solution are:

- **Model Transformation.** We designed a model transformation technique that decomposes observation models into information logic and model specific logic. In order to facilitate efficient runtime execution, the technique not only re-organizes the model information to facilitate efficient runtime access, but also pre-processes the model specific logic to facilitate code generation.
- **Unified and Efficient Runtime Store.** We proposed a runtime data-store for efficient management of runtime objects. The observation meta-model uses an object-based data model. One approach is to use an object based store, however there are performance issues in the currently available object-based datastores. Therefore, a relational datastore is desired to provide persistent support for the runtime objects. We design a unified data schema that can be used by any observation models. To improve performance, our data schema

stores the runtime objects vertically which minimizes the read/write conflicts.

- **Customized Code Generation.** We implemented a model compiler that generates Java libraries for model specific logic (i.e., expression) in a observation model. We adopted a Java virtual machine as the execution platform. This side-stepped the need to develop a home-grow evaluation engine. There are many forms of expressions in an observation model. A Java class is generated for each expression, whereby customization can be performed based on the type of expression for gaining optimal computation performance. Additionally, pre-compiled Java code contributes to better evaluation performance.

Our work is a new direction in the area of enterprise data management applications. Traditional business intelligence [1] or data warehouse solutions focus on generating reports that summarize business performance. However, they only provide services that have pull-based information delivery capability. Such services can only be invoked periodically, which cannot satisfy the realtime requirements of evaluating business performance. The continual query [6] system is able to monitor updates in areas of interest and return results whenever the updates reach specified thresholds. It provides push-enabled, event-driven, content-sensitive information delivery capabilities, which can be considered as the enabling technology for implementing situation detection. However, in the continual query system, solution providers need to specify data schemas and to program in an SQL like query language, which can be time-consuming.

Sensornet [4], the new emerging solution, processes live event data in a realtime fashion. However, this technique is not sufficient for support BPM. First, sensornet processes live event data only, while BPM processes data in both live events and persistent data stores. Second, sensornet focuses on approximate query because of the constraint of read and read only once. When monitoring business operation, such a constraint is unnecessary. Precisely processing the data and computing exact metric values is critical for business operation monitoring and is not supported by sensornet. BPM solves these problems and offers a more complete, efficient and elegant solution.

The remainder of this paper is organized as follows: Section 2 introduces the important concepts of observation modelling. Section 3 discusses the proposed hybrid approach for implementing model-driven BPM. Finally, we discuss some related work in Section 4 and provide concluding remarks in Section 5.

2 Observation Meta-Model

In this paper, the requirements of an BPM solution are captured by an observation model. The specification of observation models is given by the *observation meta-model*. In this section, we use an example of observation model (see Figure 1) to illustrate the design of observation meta-model. The meta-model is a simplified version of [3]. The meta-model describes two aspects of an observation model.

2.1 Information organization

The information in an observation model is typically constructed top-down. Formally, an observation model (see UML model Figure 2) contains a hierarchy of contexts. For example, in Figure 1, the root context contains two contexts *store* and *warehouse*, and context *store* further contains context *customer*. A context corresponds to an entity to be monitored. There are two kinds of context: statically defined (for example, *warehouse*) or dynamically created (for example, *items* in a *warehouse*). From an information organization point of view, context can be considered as a container for *metrics*. Metrics are the knowledge about business operation performance of the entity being observed. As an example, a metric can be the *stockLevel* of a *item* in *warehouse*. More specifically, a *key metric* is used to identify a context instance. For example, *storeID* is the key metric that is used to identify context *store*. *Situations* are a boolean type of metrics, and represent the gating conditions for generating *alters*. A context associates with a collection of events. Events report up-to-date status information about business operations, which are used to compute the value of metrics. It should be noted that the data structure for defining metrics and events can be primitive, structured, and single or multiple value type.

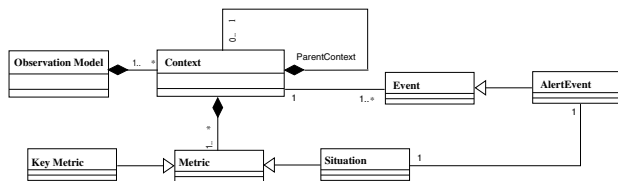


Figure 2. Information Organization in Observation Model

2.2 Operation of observation model

There are two aspects regarding the operation of an observation model (See UML model in Figure 3): (1) How the events are being processed. A event is associated

with a filter predicate and a correlation predicate. The filter predicates specify the types of events that need to be processed. Correlation predicates determine the associations between events and context instances. At runtime, an event is either correlated into existing context instances, or creates new context instance, or raise exception. It should be noted that a metric in the context is used to save the value of an event. (2) Metric network. We adopt a graphical model that captures the dependency relationships among metrics. The vertexes represents either metrics or dependencies among metrics. The edges represent the relationship between a dependency and a metric: either *inputslot* or *outputslot*. *Map expressions* are used to denote the association between a dependency vertex and the input/output metrics. As an example, in Figure 1, a map expression (map 3) is shown as: $item.stockLevel = minus(item.stockLevel, customer.order.lineItem.amount)$. In this expression, *minus* is a dependency vertex, and *item.stockLevel* and *customer.order.lineItem.amount* are metric vertexes. The *item.stockLevel* and *customer.order.lineItem.amount* are inputslots and *item.stockLevel* is an outputslot of dependency *minus*. The metrics in a map expression may belong to different contexts. For example, in above expression, two metrics are from context *item* and *customer* respectively. The relationship supported by a metric network includes functional, probabilistic, system dynamics and extensible user-defined dependency. Currently, we focus on explicit functional dependency. It should be noted that the execution of an expression can be triggered by an incoming event, value changes of a metric, or occurrence of a situation.

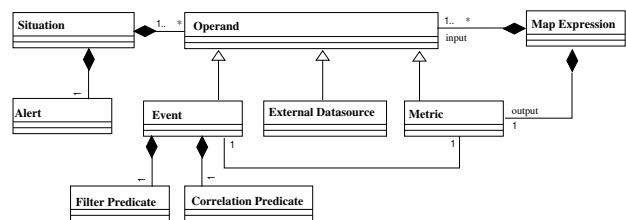


Figure 3. Operation of Observation Model

3 Hybrid Approach for Observation Model Execution

3.1 Design Rationale

If we consider our meta-model as a high level programming language, then an observation model can be considered a program. There are two approaches to executing

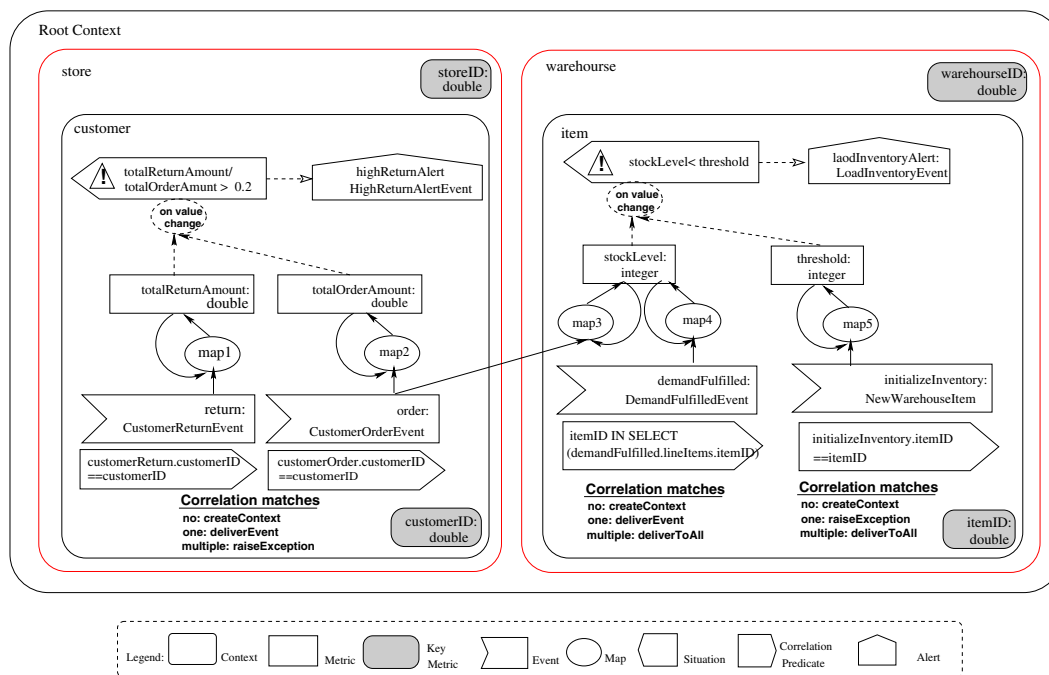


Figure 1. An Example of Observation Model

a program, namely interpreting and compiling. An interpreter is a program that implements or simulates a virtual machine using the base set of instructions of a programming language as its machine language. The source code of a programming is directed executed on by the interpreter, without generating any extra code. A compiler is a program that translates the source code of a programming language into the object code. The object code can be executed directly on the machine where it was compiled.

Similarly, the two approaches to execute an observation model are either interpreting or compiling. It should be noted that the runtime platform can be also high level, such as J2EE, etc. However, both approaches have drawbacks, given the stringent requirement of implementing BPM. We discuss adopting the interpreting approach first. The advantage of interpreting a model is that it is easy to realize the model evolution because the interpreter maintains all the model information. As we presented in the previous section, the information in the observation model is organized centrally around the contexts, however, the operation of the observation model is event-triggered. Therefore, certain information required by the runtime is scattered throughout the model. It is very costly for the interpreter to scan through the model at runtime. Also, developing an interpreter that can execute map expressions optimally becomes difficult, given that the operators in constructing map expressions can be relational, set, vector, scalar, etc. Further, the metric referenced in map expressions is not limited to the same con-

text. To locate the associated instances of a metric, the interpreter needs to navigate through the hierarchy of contexts at runtime, which may also incur performance penalties.

While adopting a compiling approach can improve execution performance by generating customized code for individual models, the model information is embedded into generated code at compiling time. The compiling approach is therefore impractical to support model evolution and hot deployment. For example, in some cases, when a context instance is in its running state, the original model must still be used to compute the metric values. Such a context session based model evolution schema is very costly when implemented by a compiling approach because a scan through all the context instances is required to check the state, as this information is maintained by the generated code.

We aim to take advantage of both approaches and propose a hybrid approach. We distinguish three kinds of application logic in an BPM solution: *common logic*, *information logic* and *model specific logic*. The common logic is defined in meta-model level, which is applied to any observation model. One example of common logic is the routine for processing events. The information logic includes information organization in the observation model. Model specific logic (e.g., map expression) is unique to each individual model.

Our hybrid approach consists of a *model transformer*, a *model compiler*, and a *model interpreter* (see Figure 4). At build time, the model transformer decomposes the ob-

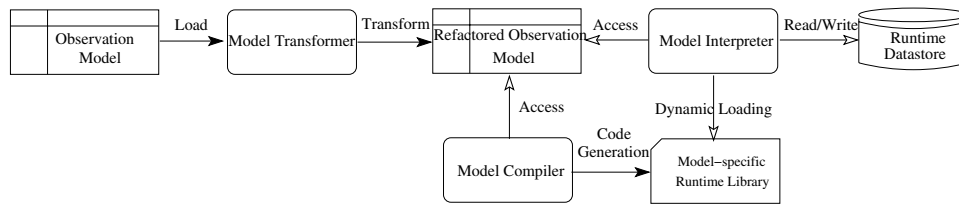


Figure 4. Hybrid Approach for Observation Model Execution

servation model into information logic and model specific logic. Also, it transforms the context-oriented model into an event-oriented model, which allows a more efficient runtime access to the model information. The compiler generates the object code for model specific logic. It should be noted that we adopted J2EE as our runtime platform. The object code of our model compiler is Java source code, which will be further compiled into Java library for runtime execution. Finally, the interpreter implements the common logic and interprets the information logic. It dynamically loads the model specific Java library to realize the operation of the observation model. In the following sections, we will present the design of these three components.

3.2 Model Transformation

On of the key step in our hybrid approach is to decompose observation models into information logic and model specific logic. Further, in order to facilitate interpretation and compilation, the model transformer possesses two main functionalities, namely (i) refactoring information logic; (ii) pre-processing model specific logic.

3.2.1 Model Refactoring

As we discussed earlier, the design of the observation meta-model aims for facilitating the creation of the observation model. However, the organization of the data is not suitable for efficient runtime access. We design the model transformer that reorganizes the model information (see data schema in Figure 5). In the refactored model, a table is created for each type of elements in an observation model. Primary and foreign keys are used to represent the cross reference among the elements. For example, by foreign key contextID in table Metric, the metrics that are belonging to a context can be located.

3.2.2 Expression Pre-processing

In observation model, the model specific logic is presented in terms of expressions in metric network. Specifically, there are four kinds of expressions: event filter expression, event correlation expression, map expression and situation

detection expression. Uniformly, these four types of expression can be denoted as:

$$f = \chi(c_1.\lambda_1, c_2.\lambda_2, \dots, c_n.\lambda_n, \gamma_1, \gamma_2, \dots, \gamma_m, \varepsilon_1, \varepsilon_2, \dots, \varepsilon_k) \quad (1)$$

χ is the operator and there are three kind of operands: metrics (λ_i), events (γ_i) and external data sources (ε_i). In case of map expression, f represents $c.\lambda$, and λ is the metric being assigned the value and the c is the context that λ belongs to. In order to facilitate the evaluation of expressions, two kinds of type-based pre-processing are performed by the model transformer at buildtime:

- **Determination of expressions that should be executed.** In the operating an observation model, the execution of an expression can be triggered by an incoming event, value changes of a metric, or occurrence of a situation. However, for sake of model creation, information in an observation model is organized centra around contexts. Without reorganizing the information, full scanning on the observation model is required in order to determinate which expression should be execution whenever occurrence of event, value changes of a metric, or situation. Therefore, it is necessary to reorganize the information and index expressions based on their execution triggers. In our solution, the refactored model maintains tables for events, metrics and situations, where each entry of event, metric or situation indexes a collection of expressions that can be triggered by the entry. For example, if a metric is an operand of map m_1 and m_2 , then the field of maps of the metric's entry of the table Metric is $\{m_1, m_2\}$. Such an index can be generated by analyzing the operands of the expressions. The computation results are saved in table Metric, Situation and Event (in the field maps and situations).
- **Determination of the navigation paths.** At runtime, the generated context instances and associated metrics form a tree structure (See Figure 6). The tree structure provides the parent-child relationship information (for example, context instance c_1^1 is the parent for context instance c_1^2) as well as contained relationship among the context instance and metrics (for example, metric λ_{11}^1 belongs to context instance

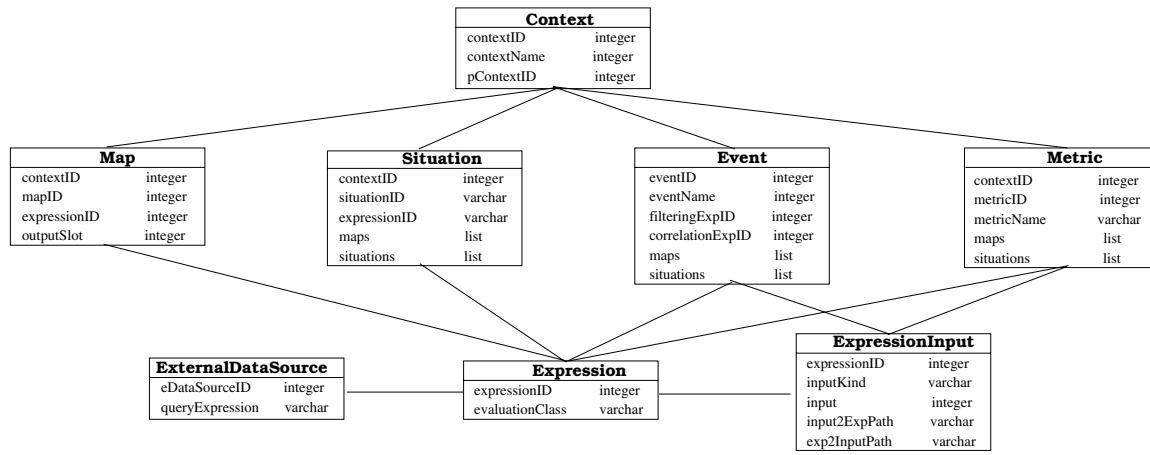


Figure 5. Simplified Refactored Observation Model Data Schema

c_1^1). In an expression, the output and operands (i.e., metrics) may belong to different contexts, wherein a path exists between the output and each operand. Based on the original context instance c that the output belongs to and destination context instance c_i that the operand metric belongs to, the navigation path can be computed as $\langle / \mu_1 / \mu_2 / \dots / \mu_k \rangle$. μ_i in the path represents a *step*. Referring to the tree structure of context instances, there are two possible directions for a step: (i) from child to parent context instance (e.g., from c_1^2 to c_1^1), where μ_i is denoted as $\dots$; (ii) from parent to child context instance (e.g., from c_1^1 to c_2^2), where μ_i is denoted as $C_i(p)$. In $C_i(p)$, C_i is the type of child context instance and p is a predicate on any metric λ_k in context c_i . The predicate p is used to identify which child context instance is in the path. When p is null, all the context instances of context type C_i are matched. In the map expression of equation (2), the output of the map expression is `Item.stockLevel` and one of the operands is `Customer.customerOrders.amount` that belongs to a different context of the map expression, then the context navigation path for locating the operand is $\langle / \dots / \dots / \text{store}() / \text{customer}(\text{item.itemID} \text{ IN } \text{SELECT}(\text{customerOrder.lineItems.itemID})) \rangle$ (as shown in Figure 6). The path consists of two child-parent steps and two parent-child steps. The first two child-parent steps reach the root context, and then step `store()` matches all the context instances with context type `store`, and finally the step `customer(item.itemID IN SELECT(customerOrder.lineItems.itemID))` matches all the context instances of `customer` that her `customerOrder` contains `lineItem` having the same `itemID` as the context instance `item`.

In our solution, the model transformer computes the context navigation path for each operand in expressions, which facilitates the searching of context instances. The computation results are saved in the `expressionInput` table (the fields `input2ExpPath` and `exp2InputPath`).

3.3 Observation Runtime Datastore

As we discussed earlier, the amount of information (i.e., context objects) manipulated by the runtime may not be able to be completely loaded into memory. Therefore, a solution to persist the context objects is required. The intuitive choice is adopting an object store (i.e., persists contexts as objects). However, it is very costly when referencing metrics attributes in map expressions: entire context objects need to be constructed in memory. In fact, in most cases, the operands in map expressions are some metric attributes instead of the whole metric. For the sake of performance and scalability, instead of adopting an object store, a relational database is used to implement the runtime datastore. Therefore, when evaluating a map expression, the runtime engine can manipulate the operands, without constructing entire context objects. When adopting a relational approach to persist contexts, mapping between context objects and relational tables is required.

An intuitive design choice for a runtime data store is to save all the context instance data as a record in a predefined table. In such an approach, the mapping between the context object and data store is simple. However, this requires creating new tables when there are changes in the information logic of a newer observation model. Considering the dynamics of business processes, evolution of the observation model may occur frequently. Therefore, the runtime data store may need to maintain a large number of tables for different versions of the observation model. In order to

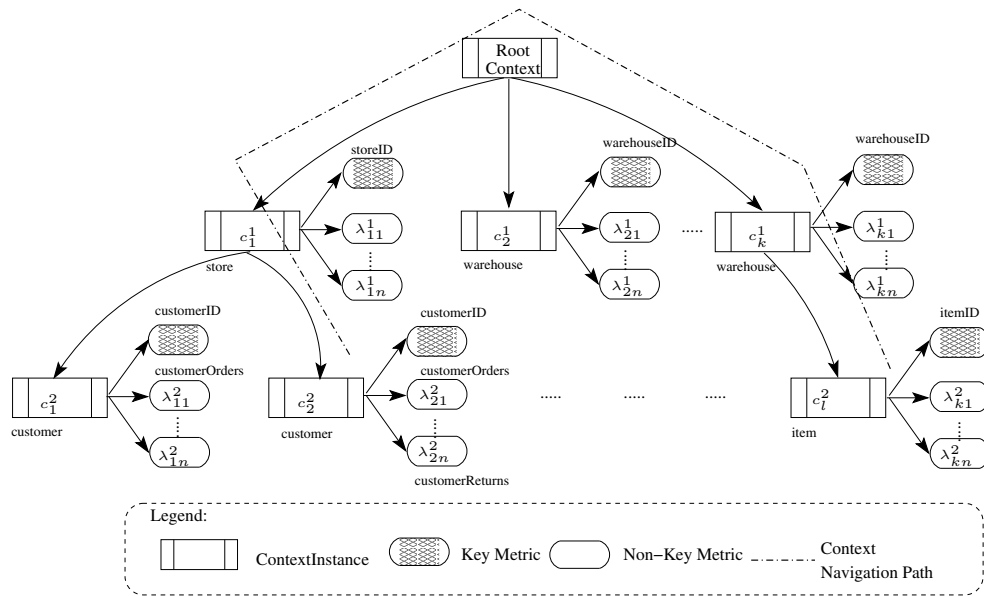


Figure 6. Context Instance Tree

overcome this limitation, we designed a unified data schema that observation models can map to. Our approach requires more complicated mappings between the context object and the data store. However, these mappings can be done at buildtime, and will not incur any performance penalty at runtime. In the rest of this section, we describe the data schema for the data store, and then we give the details on the mapping relationships between the context objects and datastore.

In our solution, the BPM server separates the organization of contexts from the data of the contexts (see Figure 7), wherein one table *ContextInstances* (C) is used to store the context instances, while another table *Values* (V) is used to store the data of contexts. It should be noted that the content of the contexts are stored vertically in table *Values*. In table *Values*, each elementary element in contexts has a record in the table and *contextInstanceID* is unique for each context instance. Using *contextInstanceID*, the records in the table can be correlated to individual context instances. Table *Dimensions* (D) is used to store the dimension information when there exists array type of data elements in contexts. By specifying *dimensionOrder* and *sequenceID*, the datastore can store any dimension array of data in a context. Further, the table *Types* (T) gives type information.

Assuming a multiple value metric *Order* contains the orders for customers, where an order consists of an array of attribute *lineItem*. The attribute itself is a structure that has two fields: *price* and *itemName*. In the runtime datastore, such a metric can be persisted as

shown in table 1. In this example, in the third row of table *Values*, *dimensionID* is 2, where in table *Dimensions*, two entries' *dimensionID* is 2: the first one's *dimensionOrder* is 1 and *sequenceID* is 2 and second one's *dimensionOrder* is 2 and *sequenceID* is 1. This indicates that position of third row in table *Values* is: 1 in first dimension, and 2 in second dimension. The first dimension represents the order sequence and the second dimension represents sequence of the *lineItem*, therefore, the row represents the attribute in first order and second *lineItem*.

3.4 Code Generation

In this subsection, we use following map expression as an example to illustrate how the code is generated for evaluating expressions.

$$c.\lambda = \chi(c_1.\lambda_1, c_2.\lambda_2, \dots, c_n.\lambda_n, \gamma_1, \gamma_2, \dots, \gamma_m, \varepsilon_1, \varepsilon_2, \dots, \varepsilon_k) \quad (2)$$

Generating code for expression evaluation consists of three steps: (1) generating code for retrieval the value of each operand; (2) generating code for executing operator χ ; (3) generating code for assigning the value to metric λ . In the rest of section, we present each step of in detail.

In first step, the model compiler generates queries to retrieve the operand's value. Here, we use metric attribute as an example of operands to illustrate our code generation solution. By specifying attributes of the metric and context navigation path, the metric operand λ_i in expression (2) can be further refined as:

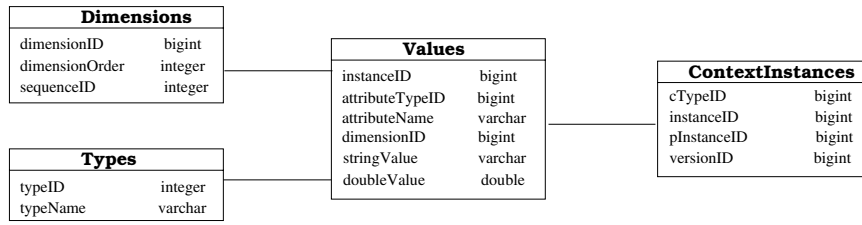


Figure 7. Runtime Datastore Data Schema

Table 1. The Snapshot of Runtime Store

instanceID	attributeTypeID	attributeName	dimensionID	stringValue	doubleValue
1	4	'order.lineItems.amount'	1	null	1
1	4	'order.lineItems.name'	1	'LED Monitor'	null
1	2	'order.lineItems.amount'	2	null	2
1	2	'order.lineItems.name'	2	'mouse'	null
1	4	'order.lineItems.amount'	3	null	1
1	2	'order.lineItems.name'	3	'printer'	null

1. Values Table

dimensionID	dimensionOrder	sequenceID
1	1	1
1	2	1
2	1	1
2	2	2
3	1	2
3	2	1

2. Dimensions Table.

typeID	typeName
1	String
2	Double
3	Boolean
4	Integer
5	Timestamp
6	Numeric

3. Types Table.

$$\lambda_i < / \mu_1 / \mu_2 / \dots / \mu_k > . a_i [d_1, d_2, \dots, d_l] \quad (3)$$

The query generation consists of two phases: (i) generating queries to retrieve the context instance; (ii) generating queries to retrieve the content of the metric attribute. We discuss how to generate queries to retrieve the context instance that the metric λ_i belongs to first. In order to selfjoin the table ContextInstances, we rename it as C_i according to each step in context navigation path:

$$\rho(C_i, C), i \in [1..k] \quad (4)$$

Then the query for retrieval target context instance is:

$$\rho(p, \pi_{C_k.instanceID}(C_1 \bowtie_{q_1} C_2 \bowtie_{q_2} \dots \bowtie_{q_{k-1}} C_k)) \quad (5)$$

This query joins all the contexts in the context path and then projects the destination context instances' instanceID. In the query, q_j ($j \in [1..k-1]$) is the equijoin predicate for C_j and C_{j+1} . The generation of equijoin predicate q_j is based on the direction of step μ_i . In case of child-parent step, predicate q_j is $C_j.pInstanceID = C_{j+1}.instanceID$, indicating C_j 's parent context is C_{j+1} . In case of parent-child step, predicate q_j is $C_j.instanceID = C_{j+1}.pInstanceID$, indicating C_j is parent context of C_{j+1} . Further, in second case, if

the predicate in the step μ is not null, then query that search for context instance that can satisfy the predicate p is:

$$\rho(V_{j+1}, \pi_{instanceID}(\sigma_{V.elementName=k.name \wedge p'}(V))) \quad (6)$$

This query selects tuples that can satisfy the predicates from table Values and projects the instanceID. In the query, p' is transformed from p by replacing the metric λ_k with $V.stringValue$ (resp. $V.doubleValue$) in predicate p , if the key metric's data type is string (resp. double). In this case, the query 5 needs to be refined as:

$$\rho(P, \pi_{C_k.instanceID}(C_1 \bowtie_{q_1} C_2 \dots \quad (7)$$

$$C_j \bowtie_{q_j} C_{j+1} \bowtie_{C_{j+1}.instanceID=V_{j+1}.instanceID} V_{j+1} \dots \bowtie_{q_{k-1}} C_k)$$

Based on query (5), this query also joins the table Values to select the right child context instances. Now we know which context instance that metric λ_i belongs to. In the following, we illustrate how to retrieve its attribute value. If the a_i 's data type is string, then the query generated for retrieval a_i is:

$$\rho(V_i, \pi_{stringValue,dimensionID}(\sigma_{V.itemName=a_i}(V \bowtie_{instanceID} P))) \quad (8)$$

This query joins the table Values with context instances that are specified by the context path. As dimension ex-

pression of a_i is $[d_1, d_2, \dots, d_l]$, the extra queries about the dimension is generated as:

$$\rho(D_{i,k}, (\sigma_{D.dimensionOrder=k \wedge D.sequenceID=d_k} D)) \quad (9)$$

$$k \in [1..l]$$

It should be noted that each above query is according to a dimension of the metric, which selects the tuple in table `dimensions` that represent the dimension is specified by the operand expression. By equijoining $d_{i,k}$ on `dimensionID` to select dimension information on same metric, we have:-

$$\rho(V_i', \pi_{stringValue}(V_i \bowtie_{dimensionID} D_{i,k})) \quad (10)$$

where V_i' represents the value of metric operand expression λ_i . It should be noted that events are considered as metrics and saved in correlated context instances. Generating code for retrieving event values can adopt the same approach for metric values. Also, an external data source is represented as a query in our model. Therefore, all the operands in the expression can be retrieved by generated or provided queries.

Now we discuss the second step: generating code for executing operators in expression. We distinguish two types of operators: (1) elementary operators are provided by the Java virtual machine; (2) advanced operators (e.g., set operator) and function can be implemented as Java methods. Once all the operands in expression are retrieved, by executing the operator, we can have the value for metric λ . Therefore, in final step, an update or insert statement is generated to set the value of metric λ in runtime datastore. It should be noted that for each expression, a Java class is generated and the Java class name is saved in table `Expression` (in the field `evaluationClass`).

3.5 Runtime Engine

In this subsection, we present the main route of the runtime engine (i.e., the model interpreter) on processing the events and computing metric values. The detail processing logic is given in flow chart (see Figure 8). In our project, we use a Message-driven Bean in J2EE to implement the runtime engine.

4 Related Work

In this section, we review some work in the model-driven architecture. Currently, model-driven architecture [7] is gaining momentum for enterprise application development. The current state of this practice employs the Unified Modeling Language (UML) as the primary modeling notation. The UML allows development teams to capture a variety

of important characteristics of a system in corresponding models. Transformation among these models can be either manual or systematic. In our solution, UML is adopted as a modeling notation for defining the meta-model of an observation model. Based on the meta-model, a GUI based modeling tool is developed to allow solution providers to visually define observation model. In regards to code generation, the Eclipse Model Framework [2] (EMF) enables code generation from UML models, wherein UML models have sufficient details about persistent and non-persistent data, business logic and presentation elements. However, in all cases, the models are the primary artifact created and manipulated by application developers, which is focused on IT implementation. In our solution, models represent the high level of business operation logic, which is defined by the solution providers. Also, code generation is adopted for implementation of model specific business logic, without any further IT level's modeling and development.

Related to MDA, Aspect-Oriented Programming [5] (AOP) complements Object-oriented programming by allowing the developer to dynamically modify the static model to create a system that can grow to meet new requirements. In AOP, the different aspects of a system's behavior are each modeled in their most natural form, and then these separate models are woven together to produce executable code. Similarly, we distinguish three aspects of observation models: common logic, information logic and model-specific logic. We do not weave these aspects together to generate executable code. Instead, we propose a hybrid approach that treats these aspects differently: common operation logic is implemented by runtime engine, information logic is interpreted by the runtime engine, while model-specific logic is compiled into Java code. Such an approach aims for efficient runtime execution and flexible model evolution.

5 Conclusions

In this paper, we present the design and implementation of Model-Driven Business Performance Management. We proposed and implemented a novel hybrid compilation-interpretation to implement the model-driven BPM. First, based on observation meta-model, we developed model transformer that refactors user defined observation models, which aims for facilitating the efficient execution of observation model. Second, we compile the operational aspect of the model, such as logic for metric computation and situation detection, into Java code. Third, we develop a runtime engine that interprets the refactored model and dynamically loads the generated code, according to the meta-model. Our framework further enables the evolution and hot deployment of the observation model. Currently, we had implemented the whole system on J2EE platform. There are

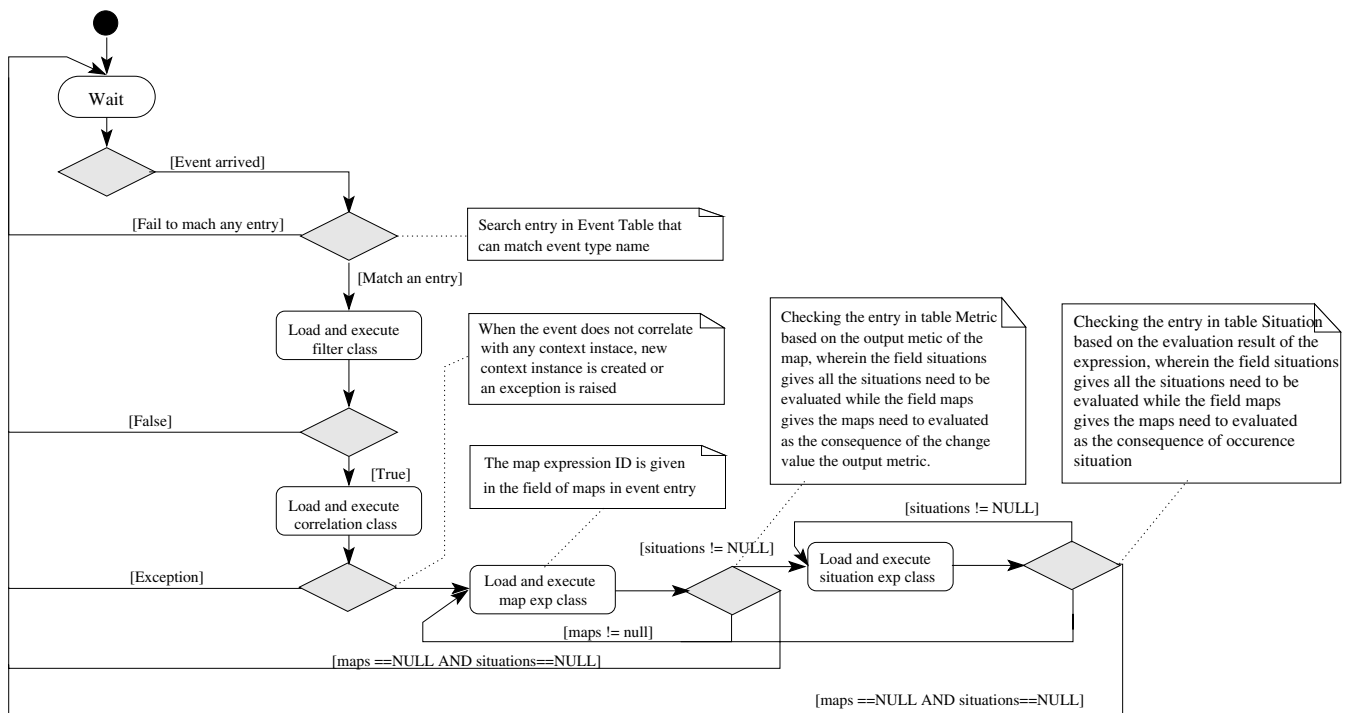


Figure 8. Main Route of Runtime Engine

several on-going customer engagement efforts to verify our solution. Our future work includes optimization of event processing, supporting metric network (i.e., probabilistic, system dynamics and extensible user-defined dependency), scalability and reliability study of the system.

References

- [1] Surajit Chaudhuri and Umeshwar Dayal. An overview of data warehousing and olap technology. *SIGMOD Record*, 26(1):65–74, 1997.
- [2] Eclipse model framework, 2005. <http://fullmoon.torolab.ibm.com/tools/emf/scripts/home.php>.
- [3] Joachim H. Frank and Ghaly Gamil Stefanos. Business Operations Metamodel, Internal Design Document, IBM, January 2005.
- [4] J. Hellerstein, W. Hong, S. Madden, and K. Stanek. Beyond average: Towards sophisticated sensing with queries. In *Workshop on Information Processing In Sensor Networks (IPSN)*, 2003.
- [5] Gregor Kiczales, John Lamping, Anurag Menhdhekar, Chris Maeda, Cristina Lopes, Jean-Marc Loingtier, and John Irwin. Aspect-oriented programming. In *Proceedings European Conference on Object-Oriented Programming*, volume 1241, pages 220–242. Springer-Verlag, Berlin, Heidelberg, and New York, 1997.
- [6] Ling Liu, Calton Pu, and Wei Tang. Continual queries for internet scale event-driven information delivery. *IEEE Knowledge and Data Engineering*, 11(4):610–628, 1999.
- [7] Model-driven architecture, 2005. <http://www.omg.org/mda/>.